



Cross-compiling (sometimes called *cross-building*) is the act of compiling code for an architecture other than that of the computer doing the compiling, e.g. building an i386 package on an amd64 laptop. For the wider topic of supporting multiple architectures, see [CategoryMultiarch](#).

Indice

1. [Background](#)
 1. [Why cross-compile?](#)
 2. [Nomenclature](#)
2. [Fix your build system](#)
3. [Build in a build environment \(recommended\)](#)
4. [Build without a build environment](#)
5. [Advanced techniques](#)
 1. [Build for Raspberry Pi OS](#)
 2. [Convert architecture names](#)
 3. [Cross-compilation-specific rules in debian/rules](#)
 4. [Simulate debhelper](#)
6. [See also](#)

Background

Why cross-compile?

Cross-compiling lets you...

- bootstrap new architectures like loong64 or riscv64
- build on beefy hardware for slow hardware
- bootstrap a customized debian
- build for platforms where the build would otherwise exhaust the 32-bit address space
- rebuild half the archive after a security incident was introduced in a package months ago
- test the time64_t transition before it happens
- reproduce arch-specific [FTBFS](#) errors on your own hardware
- develop, test and deploy everywhere with the same toolchain/OS
- 🌐 [choose boring technology](#).

Nomenclature

This page uses system-centric nomenclature. That should be familiar to users of GNU autotools, but may be confusing if you're used to the object-centric nomenclature used by other tools.

Specific terms:

build architecture

the architecture of the chroot/dpkg/compiler's executable; i.e. the architecture of the **build** system (called *host* by cmake, kernel, etc.)

host architecture

the architecture of objects produced by the compiler; i.e. the architecture of the system that will **host** these objects (called *target* or sometimes *build* elsewhere)

target architecture

if the produced object is a compiler, this is the architecture of objects *it* produces; i.e. the compilation **target** of the built program

For example, imagine you are using an amd64 laptop, cross-compiling packages for an i386 architecture, and the specific package you're cross-compiling is gcc-s390x-linux-gnu. That means amd64 is your **build** architecture, i386 is your **host** architecture, and s390x is your **target** architecture.

For more information, see [DebianMan: dpkg-architecture terms](#) and [multiarch tuples](#).

Fix your build system

Call your compiler using a variable like \$(CC) or \$(CXX), not a specific program name like gcc or g++. Either use standard variables names or document the variables. If possible, try your code in another compiler like clang.

Your project will usually be compiled *on* an amd64 PC, but can be [cross-compiled](#) for all of Debian's other architectures. Cross-compilation builds of gcc have filenames like aarch64-linux-gnu-gcc, so Debian needs to call the right command when cross-compiling.

Debian's build system sets the well-known CC and CXX environment variables when cross-compiling, but packagers can add non-standard variables in debian/rules if necessary.

Compiling your code in another compiler can be a good way to find subtle bugs. For example, `gcc` and `clang` often adapt each other's code tests, so fixing a `clang`-only warning today could avoid a `gcc` error tomorrow.

The following standard variables are available by default:

Replace...	... with
<code>ar</code>	<code>\$(AR)</code>
<code>as</code>	<code>\$(AS)</code>
<code>cc</code>	<code>\$(CC)</code>
<code>ctangle</code>	<code>\$(CTANGLE)</code>
<code>cweave</code>	<code>\$(CWEAVE)</code>
<code>g++</code>	<code>\$(CXX)</code>
<code>f77</code>	<code>\$(F77)</code>
<code>ld</code>	<code>\$(LD)</code>
<code>lex</code>	<code>\$(LEX)</code>
<code>lint</code>	<code>\$(LINT)</code>
<code>m2c</code>	<code>\$(M2C)</code>
<code>make</code>	<code>\$(MAKE)</code>
<code>cc</code>	<code>\$(OBJC)</code>
<code>pc</code>	<code>\$(PC)</code>
<code>tangle</code>	<code>\$(TANGLE)</code>
<code>tex</code>	<code>\$(TEX)</code>
<code>texi2dvi</code>	<code>\$(TEXI2DVI)</code>
<code>weave</code>	<code>\$(WEAVE)</code>
<code>yacc</code>	<code>\$(YACC)</code>

Other  [binutils commands](#) (like `strip` or `readelf`) need to be replaced with variables, but do not have standard names.

Build in a build environment (recommended)

Building packages with a [package build tool](#) is always a good idea, but is especially important when cross-compiling:

- you don't end up with lots of build-deps and foreign architecture packages in your main system
- there is no risk of corrupting your system when something gets swapped to the wrong architecture
- there are fewer conflicts due to packages lacking `Multi-Arch`: same

To cross-compile packages with [sbuild](#), [create an appropriate build environment](#) then [pass --host=<arch>](#) to sbuild.

To cross-compile packages with [pbuilder](#), [create an environment with --architecture <arch>](#), then [build packages](#) with the environment you created.

To cross-compile packages with [git-buildpackage](#), [create an environment with ARCH=<arch>](#), then pass `--git-arch=<ARCH>` to `gbp buildpackage`.

Build without a build environment

Building without a build environment may be useful in some edge cases, or to learn how the build process works. It is not recommended for general use.

First, [add multiarch support for your system](#). You may be able to skip this step if you're building a package with no architecture-specific dependencies, like a kernel or a bootloader.

Next, install [DebianPkg: build-essential](#) and `crossbuild-essential-<architecture>` (e.g. [DebianPkg: crossbuild-essential-armhf](#)).

Finally, download the source package and cross-compile it:

```
sudo apt-get build-dep --host-architecture <architecture> <pack
cd <package>-<version>
CONFIG_SITE=/etc/dpkg-cross/cross-config.<architecture> \
    DEB_BUILD_OPTIONS=nocheck \
    dpkg-buildpackage --host-arch <architecture> -Pcross,nochec
```

Advanced techniques

These may be useful in some niche cases.

Build for Raspberry Pi OS

For purposes of cross-compiling,  [Raspberry Pi OS](#) is a brand name shared by two different distributions - an *arm64* distribution based on  [Debian](#), and an *armhf* distribution based on  [Raspbian](#). Cross-compiling packages for arm64 works like any other Debian architecture, but cross-

compiling for armhf can cause strange run-time errors because [Raspbian is subtly incompatible with Debian](#).

If you need to compile packages for an *armhf* Raspberry Pi, you will need to create a chroot by hand. That means [constructing a chroot for your target distribution and architecture](#), enabling the ability to [run foreign binaries transparently](#), and adding [packages for building packages](#) to the chroot. The exact steps depend on your package build tool - for pbuilder, see [create a pbuilder Pi environment](#).

Convert architecture names

You can use `dpkg-architecture` (in [DebianMan: dpkg-dev](#)) to convert between *Debian architectures* (like `amd64`), *GNU system types* (like `x86_64-linux-gnu`), and various other [DebianMan: dpkg-architecture terms](#):

```
# Prints amd64:
dpkg-architecture -tx86_64-linux-gnu -qDEB_HOST_ARCH

# Prints x86_64-linux-gnu:
dpkg-architecture -aamd64 -qDEB_TARGET_MULTIARCH
```

For more output options, see [DebianMan: dpkg-architecture variables](#).

Cross-compilation-specific rules in debian/rules

Debian's build system handles most cross-compilation issues transparently, but complex `debian/rules` files may need to know:

1. are we cross-compiling?
2. can we run code for the host architecture?
3. should we run tests?

The difference between the first two is subtle - for example, if we cross-compile from `amd64` to `i386`, we can still run `i386` host code even though the build and host architectures are different. The difference with the third one is procedural - tests might be disabled for cross-compilation reasons, or the user might just ask to skip them. See also [user-mode emulation](#).

To check whether we are currently cross-compiling, add something like this to `debian/rules`:

```
ifneq ($(DEB_BUILD_ARCH),$(DEB_HOST_ARCH))
# We are building on one architecture, for a host of a differen
endif
```



To check whether host code can be executed, check whether [the DEB_BUILD_PROFILES environment variable](#) contains [cross](#):

```
ifneq (,$(filter cross,$(DEB_BUILD_PROFILES)))
# "cross" profile detected - cannot execute host code
endif
```

To check whether to run tests, check whether [DebianMan: the DEB_BUILD_OPTIONS environment variable](#) contains `nocheck`:

```
ifneq (,$(filter nocheck,$(DEB_BUILD_OPTIONS)))
# "nocheck" option detected - do not run tests
endif
```

Simulate debhelper

[Debhelper](#) is the recommended system for building Debian packages. You might like to step through actions it takes, so you can learn how the build process works.

You can look at the code for  [how debhelper does cross-compiling](#).

The `dpkg_architecture_value()` function returns values that can be found in the output of the `dpkg-architecture` command.

See also

-  [Qt cross-architecture development in Debian](#) by Enrico Zini
-  [Debian cross build quality assurance](#)
- [Debian automated cross-bootstrapping](#)
- [Guidelines for cross-build packaging](#)
- [Making your package ready for cross-compilation](#)
- [Debian bootstrapping](#)
- [DebianList: debian-cross mailing list](#)

[CategoryMultiarch](#)